ALTINBA■ UNIVERSITY

Faculty of Engineering and Architecture
Software Engineering Department

GRADUATION PROJECT
WEEK 3 — ARCHITECTURE AND TECHNICAL DESIGN

Project: Pharmacy Management System

Student: Deya Ghanem

Academic year: 2026–2027

Repository: https://github.com/deya-ghanem/pharmacy-management-system

Draft design package based on the Week 2 requirements. Review the architecture and understand the diagram before submission.

1. Requirements-to-Design Traceability

ID	Requirement	Design response
FR-01	Search by name/barcode	WPF search calls application service; data-access layer queries matching fields.
FR-03	Add items and calculate total	Sales UI collects lines; domain logic validates quantity and calculates total.
FR-04	Save sale and update stock	Sale, sale lines and stock deductions occur in one SQLite transaction; rollback on failure.
NFR-01	Search within 2 seconds with 5,000 records	Index fields as appropriate; test 5,000 records and time ten searches.
NFR-02	Sale and inventory consistency	Test success, cancellation, insufficient stock, database error and repeat confirmation.

2. Proposed Architecture

The initial release is a Windows desktop application on one pharmacy computer. It uses presentation, application services, domain logic and data access layers. SQLite stores data locally. No external API, cloud dependency, or multi-branch synchronization is included.

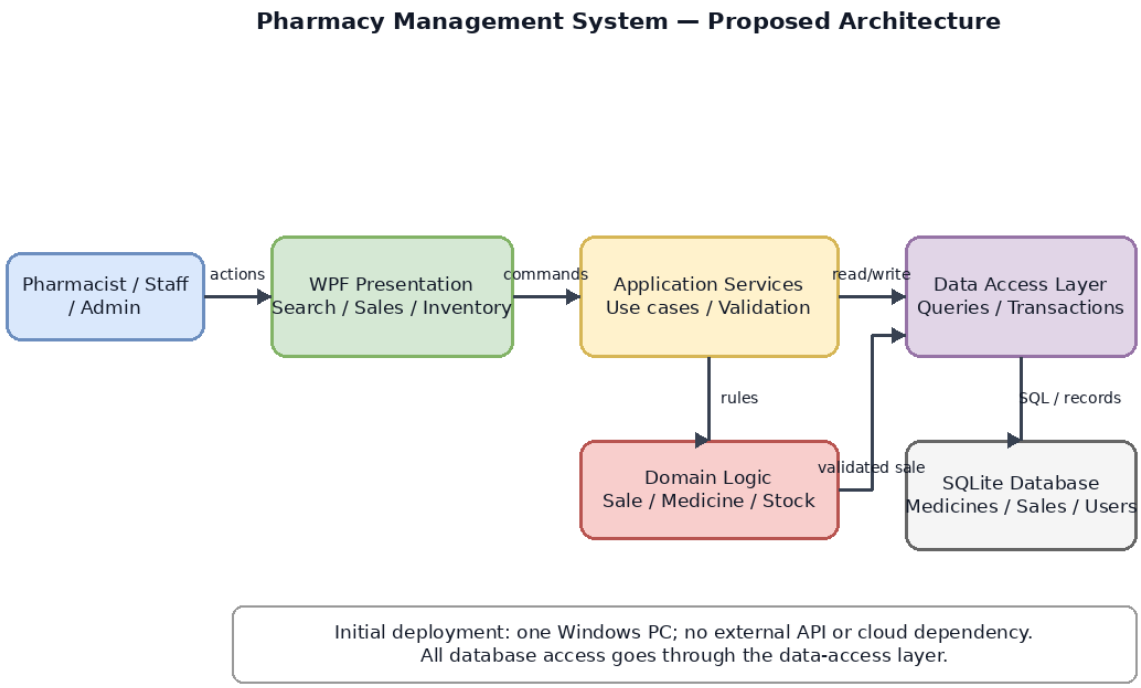


Figure 3.1. Proposed high-level architecture. Users interact with the WPF presentation layer, which sends commands to application services. Application services coordinate domain rules and data-access operations. Only the data-access layer communicates directly with SQLite. This separation supports maintainability and prevents UI code from bypassing validation and transaction handling.

3. Component Responsibilities

Component	Responsibility	Output
WPF Presentation	Search text, sale lines and user actions	Displays results, totals, confirmations and errors
Application Services	Coordinates use cases and authorization	Calls domain/data layers
Domain Logic	Validates sale quantities and rules	Validated sale and calculated total
Data Access Layer	Encapsulates SQL and transactions	SQLite reads/writes and records
SQLite Database	Persists application data	Medicines, sales, sale items and users

4. End-to-End Sale Flow

1. Staff searches by medicine name or barcode.

2. UI sends the query to the application service; data access queries SQLite.

3. Matching medicines and available quantities return to the UI.

4. Staff selects a medicine and quantity; domain logic validates stock and calculates total.

5. On confirmation, one transaction inserts the sale header and sale lines and decrements stock.

6. Success commits and displays confirmation. Failure rolls back, shows an error, and leaves stock unchanged.

5. Preliminary Data Dictionary

Entity	Key fields	Rules / notes
Medicine	MedicineId, Name, Barcode, Strength, Form, UnitPrice, QuantityInStock, ExpiryDate, IsActive	Stock ≥ 0; barcode unique when present; some details c
Sale	SaleId, CreatedAt, UserId, TotalAmount, Status	UserId identifies staff member.
SaleItem	SaleItemId, SaleId, MedicineId, Quantity, UnitPriceAtSale, LineTotal	Quantity > 0; preserve price at sale time.
User	UserId, Username, PasswordHash, Role, IsActive	Unique username; never store plaintext passwords.

Currency, tax and discount rules were not specified in Week 2 and should be confirmed before implementation. This is a preliminary entity sketch, not a final schema.

6. Technology Decisions

ADR-01 — C# + WPF on modern .NET. Alternative: Windows Forms or Python/PySide6. WPF is Windows-only and supports XAML, controls, layout and data binding, fitting the Windows-first target and a multi-screen desktop application. Reference: https://learn.microsoft.com/en-us/dotnet/desktop-wpf/overview/index

ADR-02 — SQLite. Alternative: PostgreSQL or SQL Server. SQLite supports ACID transactions, allowing sale records and stock changes to commit together or roll back together. It avoids a separate database server for the single-PC prototype. Reassess if multi-branch or concurrent-write needs arise. Reference: https://www.sqlite.org/transactional.html

These are proposed choices, not instructor-imposed constraints. The layered monolithic desktop design avoids unnecessary server complexity in the first version.

7. Security, Failure Handling and Risks

Require authentication for privileged actions and role-based authorization for edits. Validate required fields, positive quantities and non-negative stock. Store password hashes, not plaintext passwords. On database failure during sale, roll back and show an error. Backup/restore must be tested before real use.

Risk	Impact	Mitigation
Partial sale persistence / duplicate stock deduction	Incorrect sales or stock	One transaction; test forced failure, cancellation, insufficient stock and repeated confir
Slow or ambiguous search	Checkout delay or wrong medicine	Index fields; validate barcode uniqueness where present; test 5,000 records and time

8. Week 4 Prototype Acceptance Goal

Demonstrate a Windows desktop screen where a user searches a seeded medicine by name or barcode, selects it, enters a quantity, confirms a test sale, and sees a saved sale plus reduced stock. Evidence: screenshot of the UI and database output before/after. If full integration takes longer, first prove the transaction through a minimal UI or test harness.

9. Files and Next Actions

The accompanying package contains the PNG architecture diagram, editable Draw.io source, component interfaces, technology decisions, sale-flow source, and weekly progress log. Review the diagram in diagrams.net, understand every arrow, copy the architecture summary into README, update the cumulative graduation report, and push meaningful commits. Actual commit URLs remain pending until you upload the files.

References

1. Microsoft Learn, WPF overview: https://learn.microsoft.com/en-us/dotnet/desktop-wpf/overview/index

2. SQLite, SQLite Is Transactional: https://www.sqlite.org/transactional.html

3. Microsoft .NET, releases and support: https://learn.microsoft.com/en-us/dotnet/core/releases-and-support